RecipeCraft

CS3404 Mini Project — Report

1. Project Overview

RecipeCraft is a production-quality Single Page Application (SPA) built with Vue 3, TypeScript, Pinia, Vue Router, and Tailwind CSS. It consumes the DummyJSON Recipes API to deliver a polished recipe discovery and meal-planning experience.

2. Features Implemented

Core Requirements

- Data fetching from /recipes, /recipes/search, /recipes/meal-type/:type endpoints
- Search with 400ms debounce — updates results without full page reload
- Client-side filtering by cuisine, difficulty, and meal type
- Responsive layout using Tailwind Grid/Flex (mobile, tablet, desktop)
- TypeScript strict mode — zero use of *any*
- Component architecture: NavBar, HeroSection, FilterBar, RecipeCard, RecipeGrid, SkeletonCard, LoginModal
- Detail View via dynamic route /recipe/:id with full ingredient and instruction tabs

Bonus / Distinction Features

- **Authentication Simulation** — POST /auth/login, JWT stored in localStorage, Login/Logout state in Pinia authStore
- **Meal Planner (Cart)** — Global Pinia bookmarkStore persists across reloads via localStorage; adjustable servings; calorie totals
- **Bookmarks** — Bookmark/un-bookmark any recipe; persisted to localStorage
- **Dynamic Routing** — Vue Router with /recipe/:id, /bookmarks, /cart; smooth scroll restoration
- **Dark Mode** — Toggle via Tailwind dark: modifier; preference stored in localStorage
- **Skeleton Loading** — Shimmer placeholder cards during API calls
- **Page Transitions** — Fade + translate transition on route changes
- **Serving Scaler** — Adjustable servings on detail page (0.5× increments)

3. Component Architecture

The application follows a strict unidirectional data flow. State is managed in Pinia stores and flows down to components via props; events bubble up via emits.

Directory Structure

```
src/
main.ts — App entry, Pinia + Router mount
```

```
App.vue — Root shell: NavBar, RouterView, Footer
style.css — Tailwind directives + global component classes
types/index.ts — All TypeScript interfaces (Recipe, AuthUser, CartItem…)
composables/
useRecipeApi.ts — All API calls, loading/error state, client filtering
useDarkMode.ts — Reactive isDark, toggleDark, initTheme
stores/
authStore.ts — Pinia: login, logout, JWT persistence
bookmarkStore.ts — Pinia: bookmarks + meal plan cart, localStorage watcher
router/index.ts — Vue Router: /, /recipe/:id, /bookmarks, /cart
components/
NavBar.vue — Sticky nav, auth state, dark toggle, mobile menu
HeroSection.vue — Search bar, quick-tag buttons
FilterBar.vue — Debounced search input, cuisine/difficulty/meal selects
RecipeCard.vue — Recipe tile: image, badge, bookmark, stats
RecipeGrid.vue — Grid wrapper: skeleton, empty state, load-more
SkeletonCard.vue — Shimmer loading placeholder
LoginModal.vue — Teleport modal with DummyJSON auth
views/
HomeView.vue — Main page: HeroSection + FilterBar + RecipeGrid
RecipeDetailView.vue — /recipe/:id with tabs, scaler, actions sidebar
BookmarksView.vue — Saved recipes grid
CartView.vue — Meal plan list with calorie summary
```

Data Flow Diagram

Layer	Files	Responsibility
API Layer	useRecipeApi.ts composable	fetch(), error handling, client filters
State Layer	authStore, bookmarkStore (Pinia)	Auth JWT, bookmarks, cart, localStorage sync
Router Layer	router/index.ts	Route definitions, scroll behaviour, lazy loads
View Layer	HomeView, RecipeDetailView…	Page-level composition, data orchestration
Component Layer	RecipeCard, NavBar, FilterBar…	Reusable, props-driven UI atoms

4. TypeScript Strategy

Strict mode is enabled in tsconfig.json (strict: true, noUnusedLocals, noUnusedParameters). All DummyJSON response shapes are modelled in src/types/index.ts. Generic types are used for emit definitions and computed return types. No any is present in the codebase.

Key Interfaces

Recipe — Full recipe shape — id, name, ingredients[], instructions[], difficulty, rating, mealType[], …

RecipesResponse — Paginated wrapper: { recipes: Recipe[], total, skip, limit }

AuthUser — Login response: id, firstName, lastName, image, token, refreshToken

CartItem — { recipe: Recipe, servings: number } — used in bookmarkStore

FilterState — Reactive filter bag: search, cuisine, difficulty, mealType, maxCalories

5. UI / UX Design Decisions

• Custom color palette: saffron (primary), forest (success), cream (background) defined in tailwind.config.js

• CSS component layer: .btn-primary, .btn-secondary, .card, .input-field — consistent, reusable classes

• Dark mode via Tailwind dark: modifier — every component has explicit dark: variants

• Responsive grid: 1 col (mobile) → 2 col (sm) → 3 col (lg) → 4 col (xl)

• Hover micro-interactions: card lift (-translate-y-1), image zoom (scale-105), shadow deepening

• Page load animation: staggered animation-delay per card (0–420ms) using animationDelay prop

• Skeleton shimmer: CSS gradient animation on placeholder cards during loading

• Fonts: Playfair Display (display/headings) + DM Sans (body) — loaded from Google Fonts

6. DummyJSON API Endpoints Used

Endpoint	Purpose
GET /recipes?limit=&skip=	Paginated recipe listing (Load More)
GET /recipes/search?q=	Full-text recipe search
GET /recipes/:id	Single recipe detail
GET /recipes/meal-type/:type	Filter by meal type (breakfast, dinner…)
POST /auth/login	Simulate user authentication, receive JWT